Introduction aux modèles d’Architecture Logiciel

Sommaire

- Qu'est-ce que l'architecture logicielle ?
- Différents types d’architectures
- Serveurs Web
- Les API
- Systèmes modulaires
- Load Balancers et Horizontal Scaling
- Options d'hébergement
- Structures de dossiers dans le code
- Recommandations

Qu'est-ce que l'architecture logicielle ?

Architecture logicielle

- L'architecture logicielle englobe la structure de haut niveau d'un système logiciel, en se concentrant sur l’organisation des composants et leurs interactions.
- Implique de prendre des décisions de conception critiques qui ont un impact sur la structure globale du système, ses performances, son évolutivité, et la maintenabilité.
- Fournit un plan pour le système et le projet, garantissant l’évolutivité, la maintenabilité et la robustesse.
- Facilite la communication entre les parties prenantes et guide l’équipe de développement.
- Fournit les éléments clés comprennent les styles architecturaux, les motifs, indicateurs de qualité et la documentation.

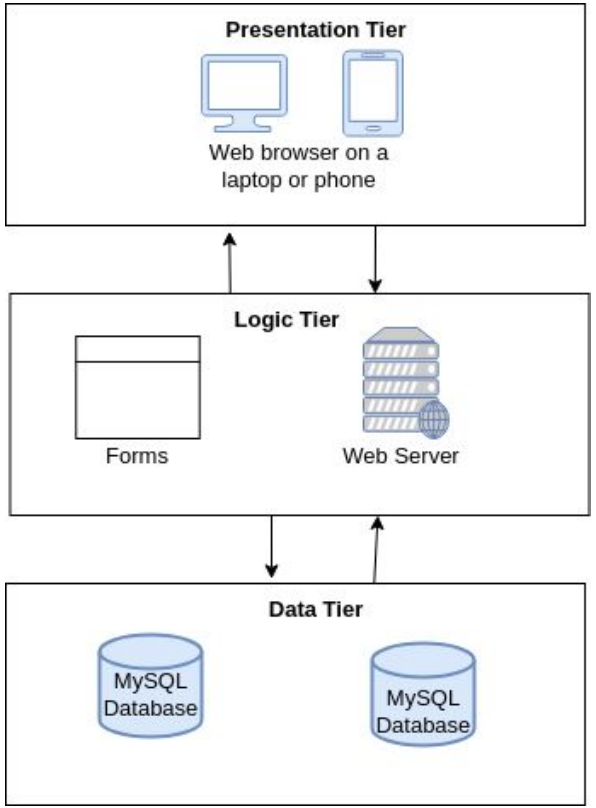
Différents types d’architectures

Architecture en couches (N-niveaux)

Description, avantages et inconvénients

- Divise l'application en couches, chacune avec des responsabilités spécifiques (par exemple, présentation, logique métier, accès aux données).
- Avantages : Séparation des besoins fonctionnelles, plus facile à gérer et à tester.
- Inconvénients : Peut être moins efficace en raison de plusieurs couches de traitement.

Architecture en couches (N-niveaux)



Architecture des microservices

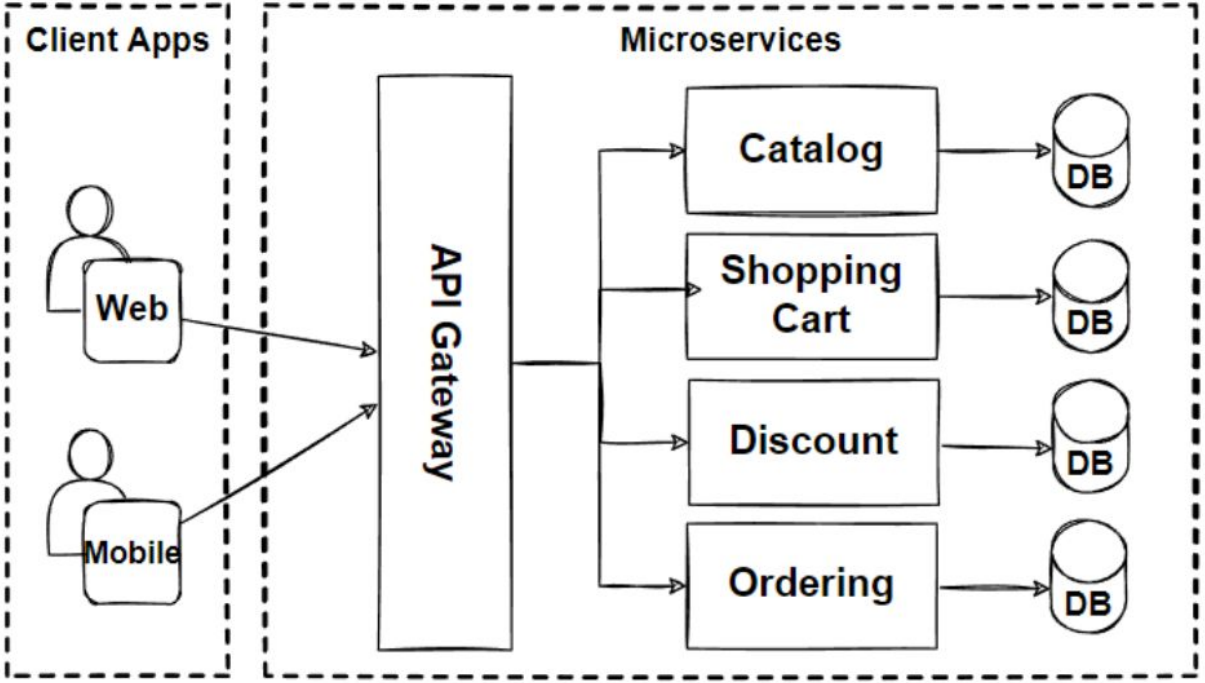
Description, avantages et inconvénients

● L'architecture des microservices décompose une application en petits services indépendants qui communiquent entre eux.

● Avantages : Haute évolutivité et flexibilité de la l’architecture technologique.

● Inconvénients : complexité accrue du déploiement et de la gestion, surcharge potentielle en termes de performances.

Architecture des microservices

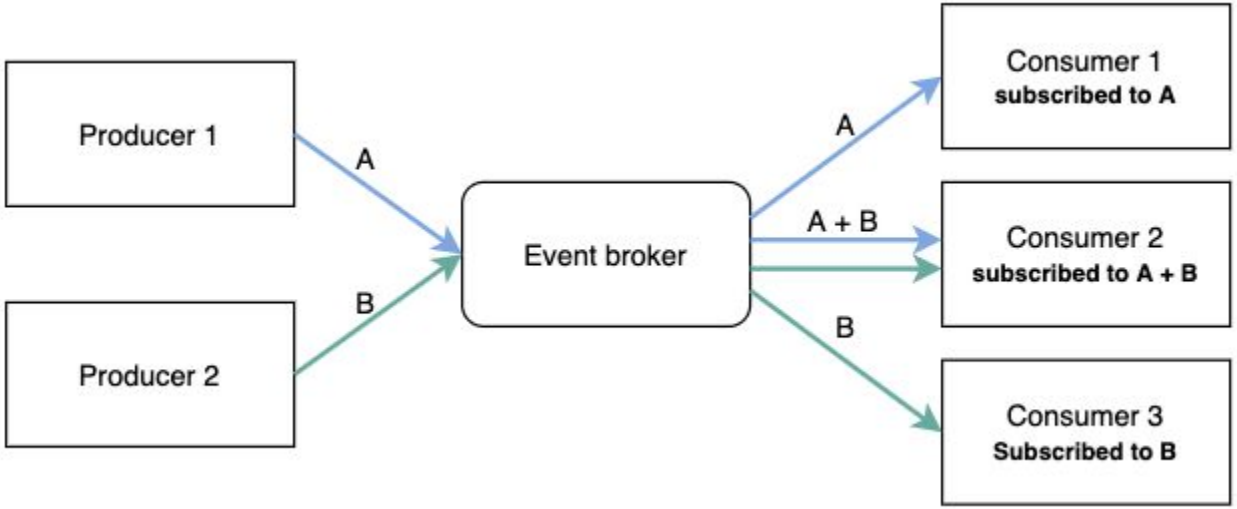


Architecture pilotée par les événements

Description, avantages et inconvénients

- L'architecture basée sur les événements utilise des événements pour déclencher la communication entre des services découplés ou des composants.
- Les avantages incluent une grande évolutivité et la promotion du découplage, permettant une évolution indépendante des services.
- Les inconvénients incluent la complexité de la gestion et du débogage, ainsi que les défis potentiels pour garantir la cohérence et la fiabilité des événements.

Architecture pilotée par les événements



Architecture orientée services (SOA)

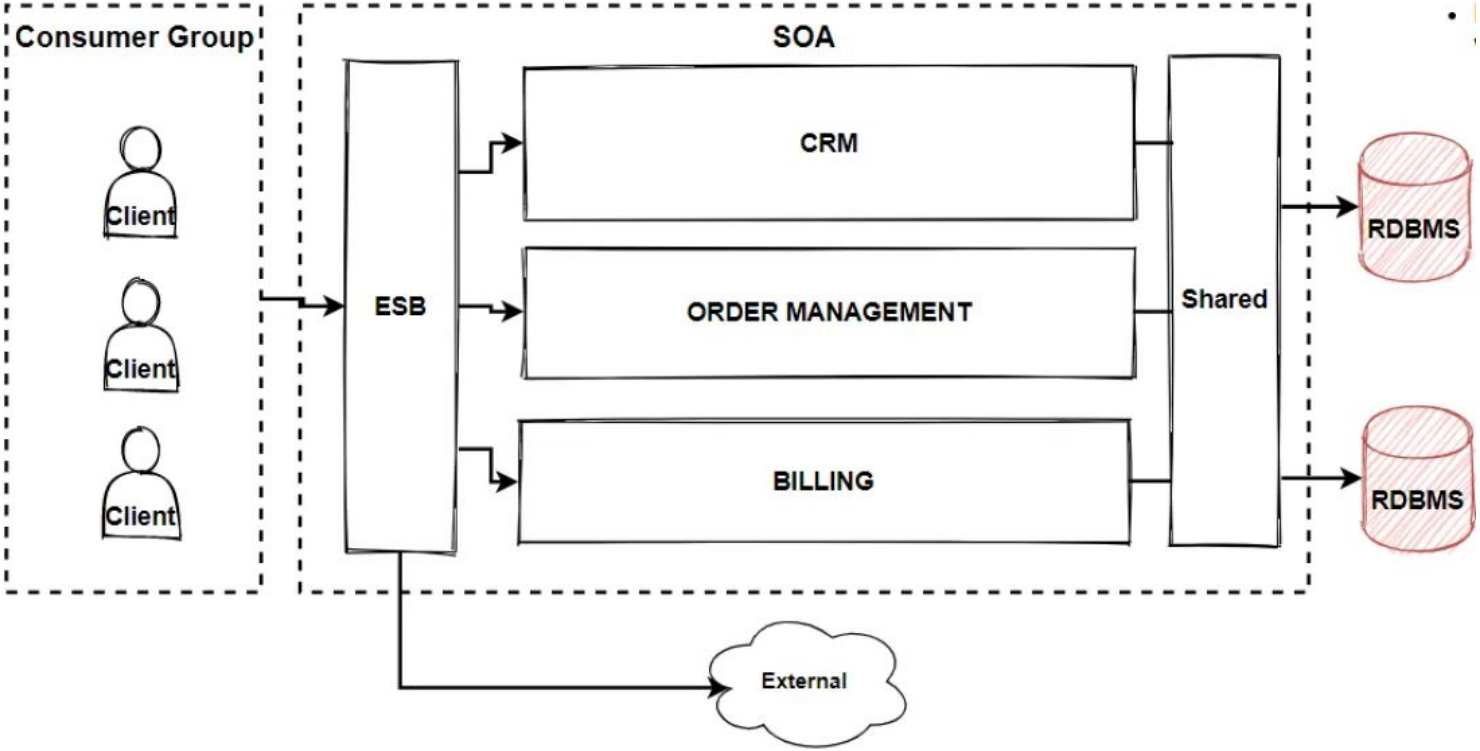
Description, avantages et inconvénients

- SOA, ou Services Oriented Architecture, est un style architectural dans lequel les services sont fournis à d'autres composants par des composants d'application, via un protocole de communication sur un réseau.

- Les avantages de la SOA incluent la réutilisabilité des services, la facilité d'intégration avec les systèmes existants et une évolutivité et une flexibilité améliorées.

- Les inconvénients incluent la possibilité de créer des systèmes étroitement couplés s'ils ne sont pas gérés correctement, la complexité de la gestion des services et une éventuelle surcharge de performances due à la communication réseau.

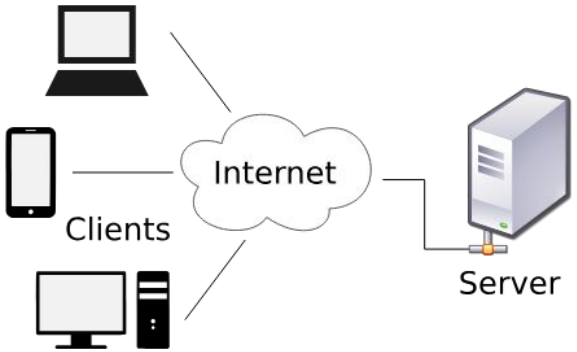
Architecture orientée services (SOA)



Serveurs Web

Modèle client-serveur: Aperçu et exemples

- Le modèle client-serveur répartit les tâches entre les fournisseurs de ressources (serveurs) et les demandeurs (clients).
- Les exemples incluent les navigateurs Web demandant des pages Web aux serveurs et les applications mobiles demandant des données aux services backend.
- Les clients et les serveurs peuvent être développés, hébergés et exploités indépendamment.

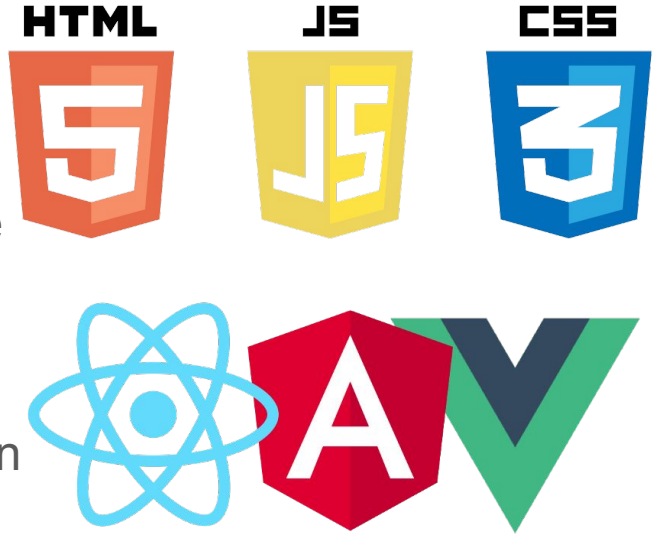


Modèle client-serveur: Serveurs avec état ou sans état

- Serveurs avec état : conserve l'état du client sur plusieurs requêtes (par exemple, sessions dans des applications Web).
- Serveurs sans état : chaque requête contient toutes les informations nécessaires à son traitement (par exemple, les API RESTful).
- Différence clé : les serveurs avec état assurent le suivi des interactions des clients, tandis que les serveurs sans état traitent chaque requête de manière indépendante.

Développement Frontend

- Technologies : HTML, CSS, JavaScript et frameworks comme React, Angular, Vue.js.
- Responsabilités : Interface utilisateur, expérience utilisateur, validation côté client et interaction avec les services backend.
- Défis : Compatibilité des navigateurs, optimisation des performances, responsive design.



Développement Backend

- Technologies : Node.js, Express, Django, Spring Boot, bases de données comme MySQL, MongoDB.
- Responsabilités : Logique métier, stockage et récupération de données, authentification et autorisation, validation côté serveur.
- Défis : évolutivité, sécurité, gestion de base de données.

Les API

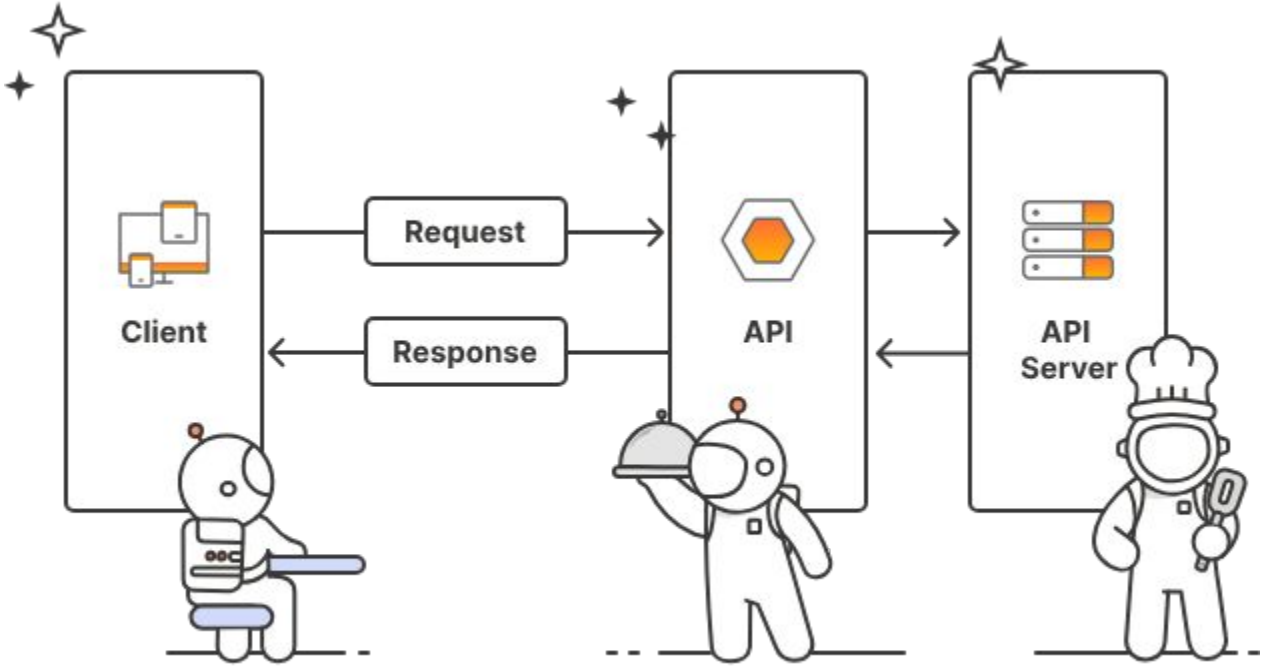
Qu'est-ce qu'une API ?

Une API, ou Application Programming Interface, est un ensemble de règles définies qui permettent à différentes entités logicielles de communiquer entre elles.

En exposant des points de terminaison et des méthodes spécifiques, les API facilitent l'intégration de différents systèmes, leur permettant de partager des données et des fonctionnalités.

Dans l'architecture logicielle moderne, les API sont cruciales pour créer des applications évolutives et modulaires.

Qu'est-ce qu'une API ?



API REST

REST (Representational State Transfer) utilise des méthodes HTTP standard telles que GET, POST, PUT, DELETE.

C'est sans état, ce qui signifie que chaque demande contient toutes les informations nécessaires.

Les réponses sont généralement des données aux formats JSON ou XML.

API SOAP

SOAP (Simple Object Access Protocol) est un protocole d'échange d'informations structurées.

Il utilise XML pour les messages et prend en charge des transactions plus complexes.

Il est connu pour sa robustesse et sa gestion intégrée des erreurs.

GraphQL

GraphQL est un langage de requête pour les API.

Il permet aux clients de demander exactement les données dont ils ont besoin, ce qui le rend plus efficace.

Il prend en charge les requêtes dynamiques et est connu pour sa flexibilité.

Sécurité des API

Authentification (Authentication)

S'assurer que l'API peut vérifier l'identité du client.

Les méthodes courantes incluent OAuth pour l'authentification basée sur des jetons et les clés API pour des cas d'utilisation plus simples.

Sécurité des API

Autorisation (Authorization)

S'assurer que le client authentifié a le droit d'accéder à la ressource demandée.

Cela implique souvent des rôles et des autorisations pour contrôler les niveaux d'accès (RBAC).

Sécurité des API

Limitation du débit (Rate Limiting)

Prévenir les abus en limitant le nombre de requetes qu'un client peut effectuer dans une période donnée.

Cela permet d’éviter que les API ne soient submergées et de garantir une utilisation équitable.

Sécurité des API

Validation des données et Désinfection (Data Validation and Sanitization)

S'assurer que les données envoyées à l'API sont valides et sûres.

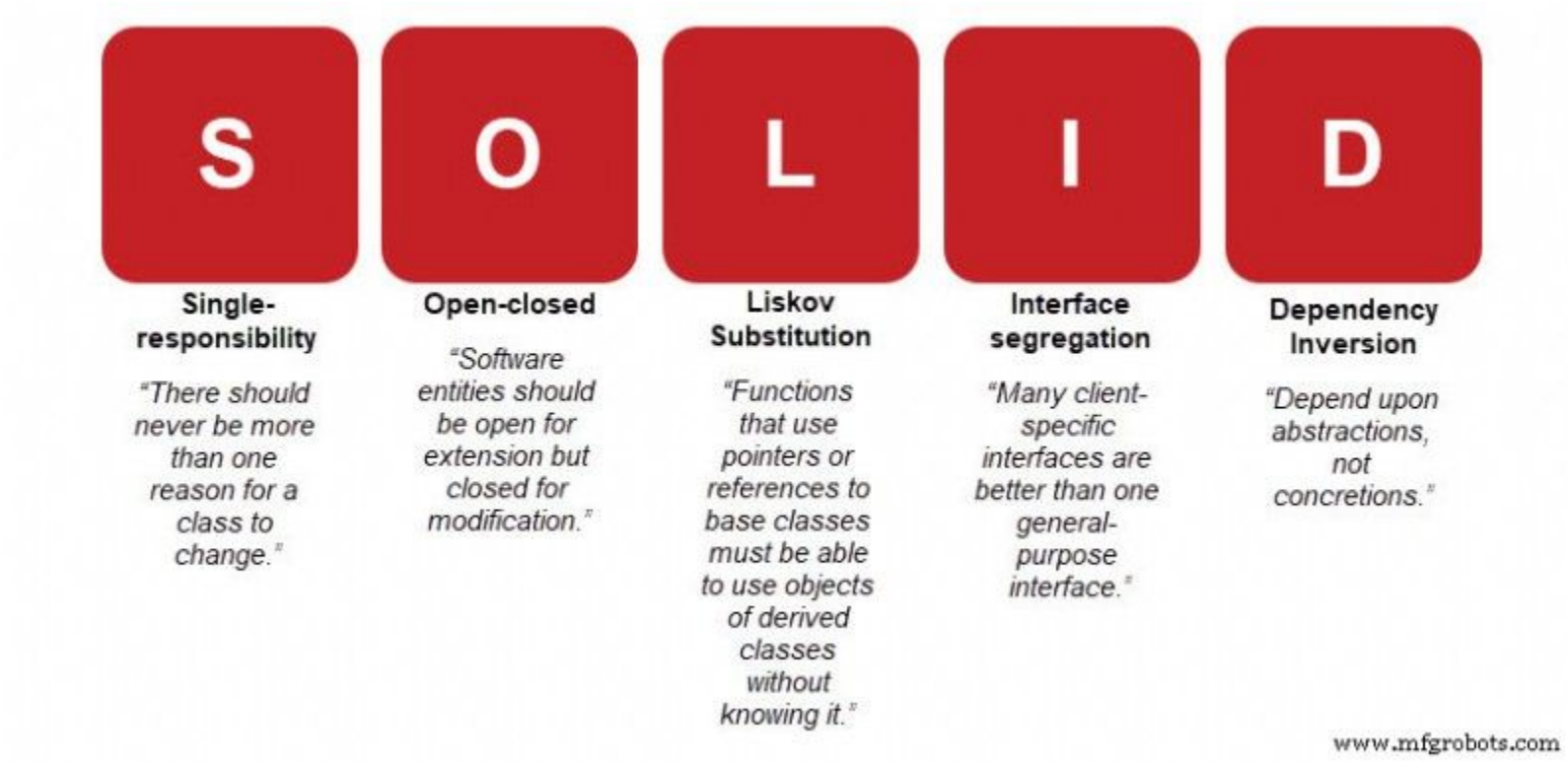
Cela implique de vérifier les types et formats de données corrects et de supprimer les entrées potentiellement dangereuses.

Systèmes modulaires

Qu'est-ce que la modularité ?

- La modularité est un principe de conception qui divise un système logiciel en modules distincts et indépendants, chacun ayant une responsabilité spécifique.
- Il favorise la séparation des besoins fonctionnelles, rendant le système plus facile à comprendre, à développer et à maintenir.
- Les avantages de la modularité incluent une maintenabilité, une évolutivité et une réutilisation améliorées.
- Les systèmes modulaires sont plus faciles à tester et à déboguer puisque les problèmes peuvent être isolés au sein de modules individuels.
- Permet un développement parallèle par différentes équipes, améliorant ainsi l'efficacité du développement.

Principes de conception de Systèmes Modulaire



Principe SOLID

Principe de responsabilité unique (SRP)
Single Responsibility Principle

Chaque module doit en avoir un, et une seule raison de changer.

Cela rend le système plus facile à comprendre et maintenir.

Principe SOLID

Principe ouvert/fermé (OCP)
Open/Closed Principle

Les modules doivent être ouverts pour extension mais fermé pour modification.

Il permet d'ajouter de nouvelles fonctionnalités sans changer code existant.

Principe SOLID

Principe de substitution de Liskov (LSP)
Liskov Substitution Principle

Les sous-types doivent être substituables à leur types de base sans altérer l’exactitude du programme.

Cela garantit que les classes dérivées étendent la classe de base sans modifier son comportement.

Principe SOLID

Principe de ségrégation d'interface (ISP)
Interface Segregation Principle

Les clients ne doivent pas être obligés de dépendre sur les interfaces qu’ils n’utilisent pas.

Cela signifie créer des interfaces spécifiques pour chaque ensemble de fonctionnalités.

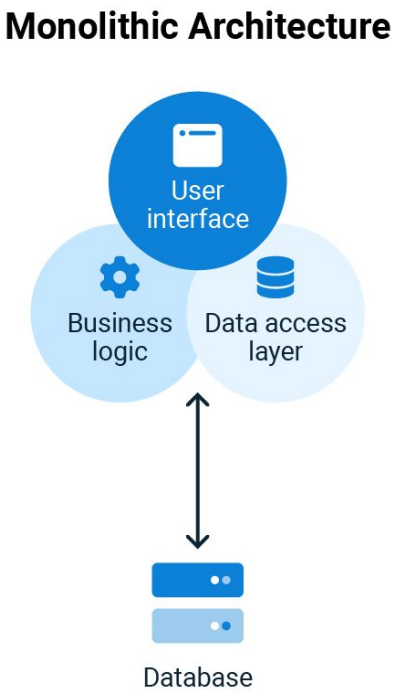
Principe SOLID

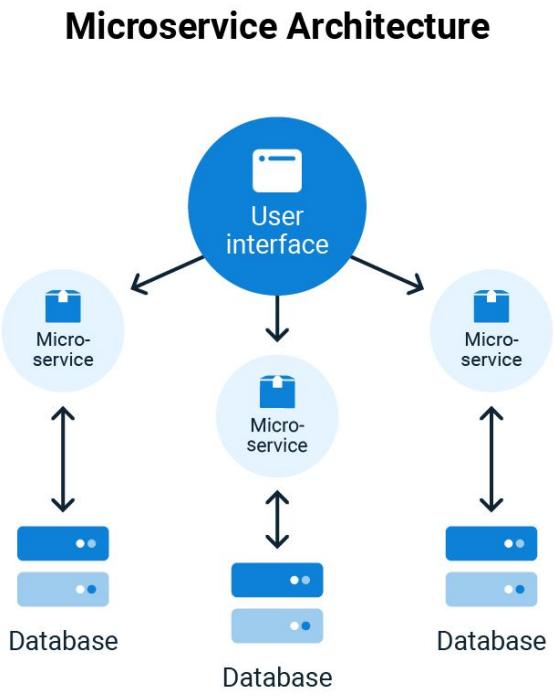
Principe d'inversion de dépendance (DIP)
Dependency Inversion Principle

Les modules de haut niveau ne doivent pas dépendent de modules de bas niveau. Les deux devraient dépendre d’abstractions.

Ce favorise le couplage lâche et la flexibilité.

Architecture monolithique ou microservices





Architecture monolithique ou microservices

Architecture monolithique

- Tous les composants sont regroupés dans une seule unité.
- Avantages : Développement et déploiement simples, tests faciles dans leur ensemble, performances optimisées.
- Inconvénients : problèmes d'évolutivité, difficulté à gérer une base de code volumineuse, flexibilité limitée dans l'adoption de nouvelles technologies.

Architecture monolithique ou microservices

Architecture de microservices

- L'application est décomposée en petits services autonomes.
- Avantages : évolutivité, flexibilité dans les choix technologiques, isolation améliorée des bugs.
- Inconvénients : complexité accrue dans la gestion des communications interservices, déploiement et surveillance complexes, problèmes de cohérence des données.

Bonnes pratiques en architecture microservices

Découverte de services (Service Discovery)

Les mécanismes de découverte de services aident à localiser les services dans une architecture de microservices. Cela garantit que les services peuvent se trouver dynamiquement et communiquer entre eux sans avoir à coder en dur les adresses.

Implémentez des outils de découverte de services comme Eureka ou Consul. Assurez-vous que tous les services s'enregistrent auprès du serveur de découverte. Configurez les clients pour interroger le serveur de découverte pour connaître les emplacements de service.

Bonnes pratiques en architecture microservices

Passerelle API (API Gateway)

Une passerelle API sert de point d'entrée pour toutes les demandes des clients. Il gère le routage des requêtes, la composition et la traduction du protocole, simplifiant ainsi l'interaction côté client avec les services backend.

Configurez une passerelle API comme NGINX, KONG, AWS API Gateway. Acheminez les requêtes des clients via API Gateway. Implémentez la sécurité et la limitation du débit au niveau de la passerelle.

Bonnes pratiques en architecture microservices

Modèle de disjoncteur (Circuit Breaker Pattern)

Le modèle de disjoncteur empêche les pannes en cascade dans les microservices en détectant les pannes et en court-circuitant les requêtes vers les services défaillants, permettant ainsi au système de récupérer correctement.

Intégrez des librairies de disjoncteurs comme Hystrix ou Resilience4j. Surveillez l’ état du service et déclenchez les disjoncteurs si nécessaire. Enregistrez et analysez les événements des disjoncteurs pour identifier et résoudre les problèmes.

Bonnes pratiques en architecture microservices

Journalisation centralisée (Centralized Logging)

La journalisation et la surveillance centralisées sont cruciales pour la gestion des microservices. Il aide à suivre les demandes, à déboguer les problèmes et à garantir la santé du système.

Déployez des outils de journalisation comme ELK Stack (Elasticsearch, Logstash, Kibana) ou Prometheus. Centralisez les journaux de tous les services dans un seul système. Configurez des tableaux de bord et des alertes pour surveiller les performances et les erreurs du système.

Backend For Frontend (BFF)

Le modèle BFF introduit un service backend distinct adapté aux besoins d'un utilisateur spécifique ou interface utilisateur spécifique. Cela simplifie la logique côté client et optimise les interactions back-end.

Avantages

BFF simplifie la logique côté client en éliminant les complexités du back-end, optimise les interactions back-end pour les besoins spécifiques des clients et améliore les performances et l'évolutivité en réduisant les transferts de données inutiles.

Conseils de mise en œuvre

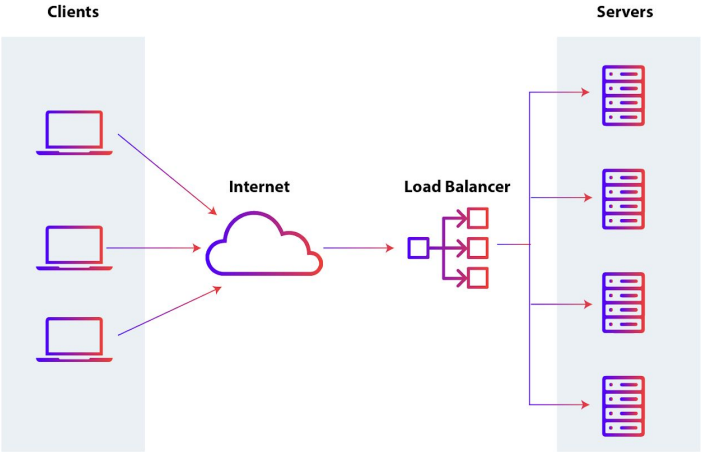
Définissez des limites claires entre les services BFF et backend, garantissez des pratiques de sécurité cohérentes entre BFF et backend, et surveillez et enregistrez les performances de BFF séparément pour identifier les goulots d'étranglement.

Load Balancers et Horizontal Scaling

Load Balancers

Équilibreurs de charge

Les équilibreurs de charge répartissent le trafic réseau entrant sur plusieurs serveurs pour garantir la fiabilité et les performances.



Types d’équilibreurs de charge

Les équilibreurs de charge peuvent être basés sur du matériel ou des logiciels (par exemple, Nginx, HAProxy).

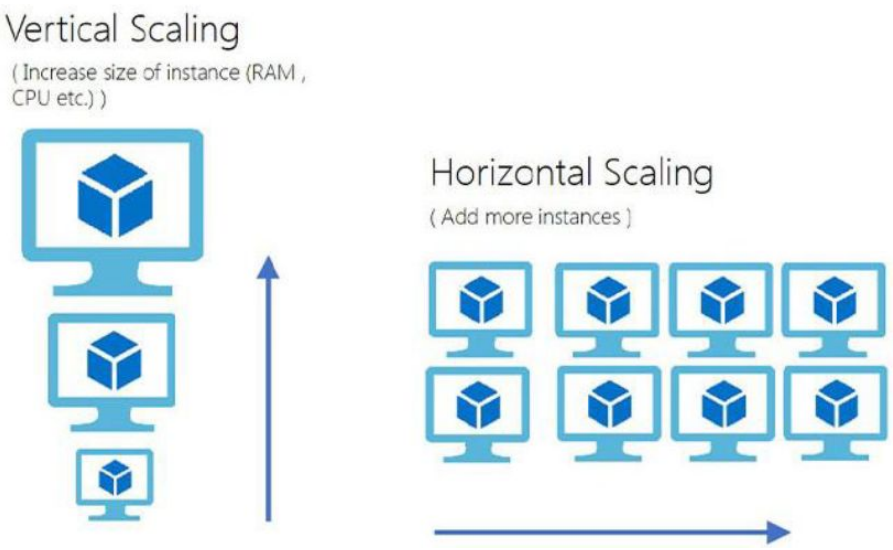
Algorithmes d'équilibrage de charge

Les algorithmes courants incluent le round-robin, les moindres connexions et le hachage IP, chacun avec des stratégies de distribution de trafic différentes.

Horizontal Scaling

La mise à l'échelle horizontale implique l'ajout de plusieurs instances de serveur pour gérer une charge accrue.

Il peut être implémenté à l’aide de plates-formes d’orchestration de conteneurs telles que Kubernetes ou de groupes de mise à l’échelle automatique dans le cloud.



Options d'hébergement

Hébergement sur site

On-Site Hosting / On premise

● L'hébergement sur site implique la configuration et la maintenance de serveurs physiques sur votre propre site au lieu de les louer auprès de fournisseurs tiers.

● Avantages : Offre un contrôle total sur les configurations matérielles, les mesures de sécurité et le respect de réglementations strictes en matière de protection des données.

● Défis : coûts de configuration initiaux élevés pour l’achat de matériel, dépenses de maintenance et d’exploitation continues et évolutivité limitée.

Fournisseurs de serveurs traditionnels

Traditional Server Providers

● Les fournisseurs de serveurs traditionnels proposent des ressources matérielles louées pour l'hébergement d'applications et de services Web.

● Avantages : Dépenses d'investissement réduites car le matériel est loué, flexibilité d'augmentation ou de réduction en fonction de la demande.

● Des exemples de fournisseurs de serveurs traditionnels incluent Hostinger, Bluehost et DigitalOcean.

● Défis : Dépendance vis-à-vis du fournisseur pour les problèmes matériels, latence potentielle en fonction de l’emplacement des centres de données du fournisseur.

Hébergement cloud

Cloud Hosting

Principaux fournisseurs et modèles :

- Principaux fournisseurs : AWS, Google Cloud Platform, Microsoft Azure
- Hébergement Cloud traditionnel : location de machines virtuelles spécifiques
- Elastic Cloud Hosting : mise à l'échelle automatique des ressources en fonction de la demande
- Informatique sans serveur (Serverless Computing) : exécuter du code sans provisionner ni gérer de serveurs

Hébergement cloud

Cloud Hosting

Avantages et défis :

• Avantages : modèle de tarification à l'utilisation, haute disponibilité, intégration avec divers services

• Défis : gestion des coûts pour éviter les dépenses imprévues, sécurité et conformité dans les environnements multi-locataires, problèmes de dépendance vis-à-vis des fournisseurs.

• Gérer et optimiser l'allocation des ressources pour éviter le surprovisionnement ou la sous-utilisation.

Structures de dossiers dans le code

Dossier Unique

Single Folder Structure in Code

● Une structure de dossiers unique est une approche organisationnelle simple dans laquelle tous les fichiers de code sont stockés dans un seul répertoire.

● Cette structure est couramment utilisée pour les petits projets ou lorsqu’un projet vient tout juste de démarrer.

● Les problèmes liés à une structure de dossiers unique incluent des difficultés de gestion et de navigation dans la base de code à mesure qu'elle grandit.

● Cela peut conduire à un environnement encombré dans lequel la recherche et la maintenance de fichiers spécifiques deviennent difficiles.

● Cette structure manque souvent de séparation claire des besoins fonctionnelles, ce qui rend plus difficile la mise en œuvre et le test des composants individuels.

Dossiers en couches

Layered Folder Structure

- Une structure de dossiers en couches organise le code en couches distinctes, chacune avec des responsabilités spécifiques.
- Exemple de structure : couche d'application, couche de routes, couche de contrôleurs, couche de modèles, couche de persistance.
- Avantages : organisation claire, gestion et évolutivité plus faciles, et testabilité améliorée.

Dossiers basée sur les fonctionnalités

Feature-Based Folder Structure

- Organise le code par fonctionnalités ou modules.
- Regroupe tous les fichiers liés à une fonctionnalité spécifique.
- Chaque répertoire de fonctionnalités contient des composants tels que des contrôleurs, des modèles, des vues et des routes.

Dossiers basée sur les fonctionnalités

Avantages :

• Structure modulaire et évolutive.

• Facilite la collaboration en équipe en permettant un travail indépendant sur différentes fonctionnalités.

Inconvénients :

• Peut conduire à une duplication de code si les fonctionnalités communes ne sont pas extraites.

Dossiers de conception pilotée par domaine (DDD)

Domain-Driven Design (DDD) Folder Structure

- Organise le code autour du modèle de domaine et de la logique métier.
- S'aligne sur le domaine métier, facilitant ainsi la gestion des logiques complexes.
- Favorise la modularité et la séparation des besoins fonctionnelles.

Dossiers de conception pilotée par domaine (DDD)

Avantages

- Aligne le code avec la logique métier, ce qui le rend plus facile à comprendre.
- Favorise la modularité, facilitant la maintenance et l'évolutivité.

Inconvénients

- Peut être complexe à configurer et à comprendre pour les nouveaux développeurs.
- Nécessite une compréhension approfondie du modèle de domaine.

Recommandations

Commencez par les bases

Maîtrisez les concepts fondamentaux tels que les structures de données, les algorithmes et les principes de programmation de base. Ce sont les éléments constitutifs de tout le reste.

Comprendre les modèles de conception

Apprenez les modèles de conception courants tels que Singleton, Factory et Observer. Ces modèles résolvent des problèmes récurrents et améliorent la qualité du code.

Explorer la documentation

Soyez à l’aise avec la lecture et la rédaction de documentation. Une bonne documentation vous aide à comprendre les systèmes existants et à communiquer efficacement vos propres conceptions.

Pratique, pratique, pratique

Créez des projets, contribuez à l'open source ou participez à des défis de codage. L'expérience pratique est inestimable pour l'apprentissage et la rétention.

Rechercher des commentaires

N'ayez pas peur de demander des révisions de code ou du mentorat. Les commentaires constructifs vous aident à développer et à améliorer vos compétences en codage.

Restez curieux

La technologie est en constante évolution. Continuez à apprendre de nouveaux langages, frameworks et outils pour rester à jour et polyvalent dans votre carrière.

Conclusions

Résumé

- L'architecture logicielle est cruciale pour la structuration, la mise à l'échelle et la maintenance des systèmes logiciels, impliquant des éléments clés tels que les styles architecturaux, les modèles et les attributs de qualité.
- Différents styles architecturaux tels que les couches, les microservices, les événements et la SOA ont leurs propres avantages et inconvénients.
- Le modèle client-serveur sous-tend de nombreuses architectures Web, avec des distinctions entre les rôles frontend et backend, et l'importance des API dans la communication logicielle.
- La modularité améliore la maintenabilité, l'évolutivité et la réutilisabilité, avec des principes tels que SRP, OCP et DIP guidant la conception des systèmes modulaires.
- Les options d'hébergement vont de l'hébergement sur site à l'hébergement cloud, chacune présentant des avantages et des inconvénients uniques.

Webographie

Webographie

- Pour des informations complètes sur l’architecture logicielle, visitez : https://alaolawal.gumroad.com/l/larchitecturelogiciel
- https://microservices.io/ pour plus d'informations sur les architectures de microservices et leurs associées
- https://refactoring.guru/ pour en savoir plus sur les modèles de conception et certaines meilleures pratiques de conception de logiciels
- https://bytebytego.com/ pour des articles sur la conception et l'architecture de systèmes
- L'architecture logicielle par Raju Gandhi, Mark Richards et Neal Ford
- Fondamentaux de l'architecture logicielle, une approche d'ingénierie par Mark Richards et Neal Ford
- Architecture propre et code propre par Robert C. Martin